

Git pour le labo 1

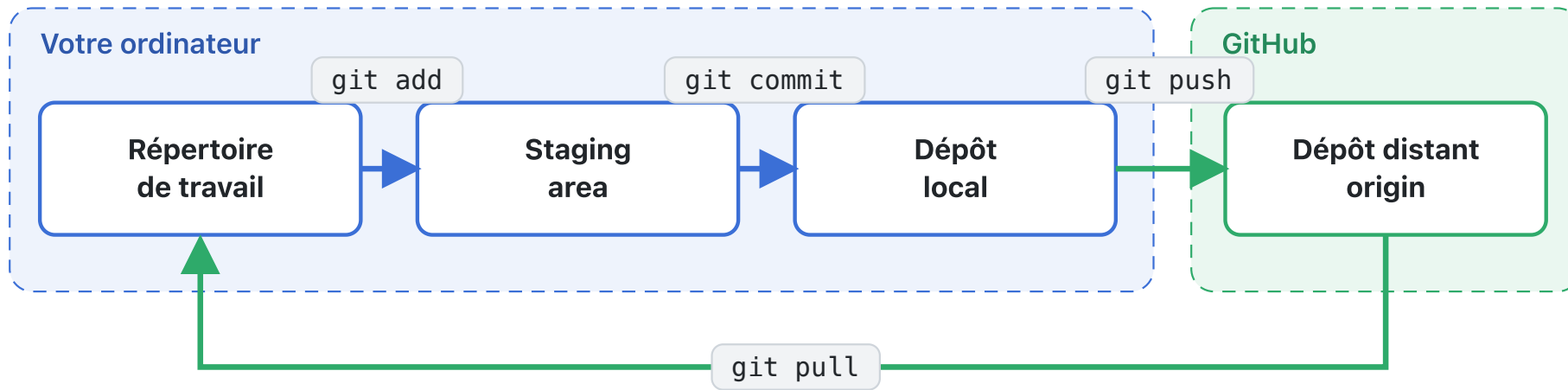
Branches, pull requests et ce qui coince

Olivier Tischhauser

Automne 2026

Rappel : trois endroits

Un commit reste sur votre ordinateur tant que vous ne l'avez pas poussé.



- Un **commit** = une photo du projet, avec un message
- Le commit est sur **votre** ordinateur tant que vous n'avez pas fait **push**
- **pull** ramène les commits des autres

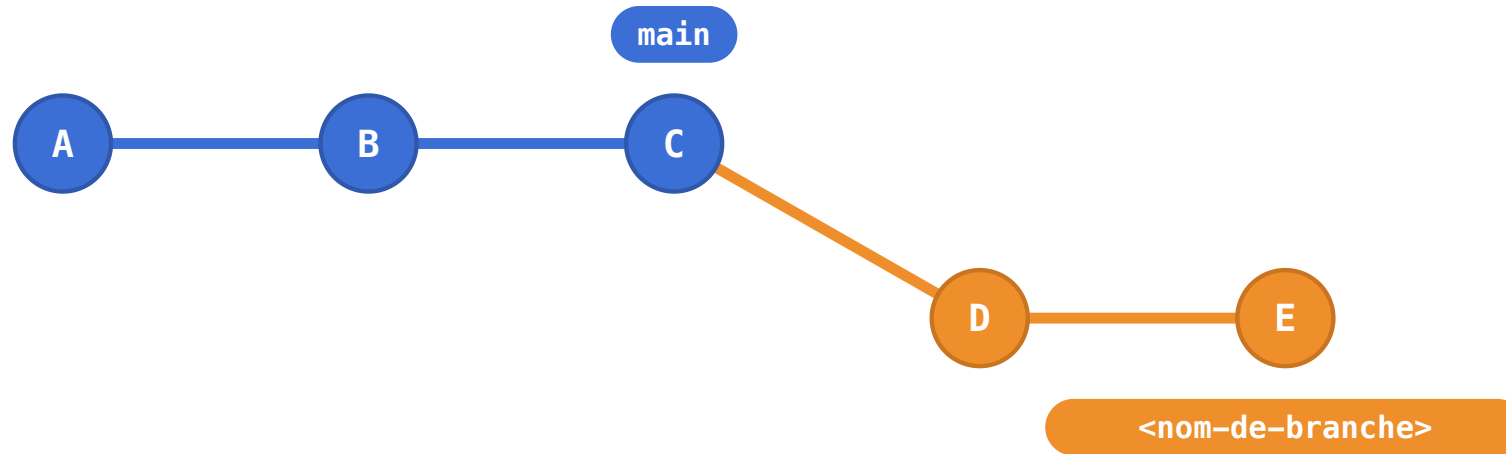
Git répond à trois questions

Question	Commande	Dans CLion
Qu'est-ce que j'ai changé ?	<code>git status</code> , <code>git diff</code>	onglet Commit
Qui a fait quoi, quand ?	<code>git log</code>	Git > Show Git Log
Comment revenir en arrière ?	<code>git checkout <fichier></code>	Git > Rollback

- Chaque commit a un identifiant unique
- On ne perd rien de ce qui a été commité

Une branche

git checkout -b crée la branche à partir de main. Ses commits D et E ne touchent pas main.



- Une branche est un **pointeur** sur un commit : la créer ne copie rien
- Chacun travaille sur **sa** branche, **main** reste propre
- Nom de branche : ce qu'elle apporte, par exemple **presentation-ana**

La pull request

Pourquoi ?

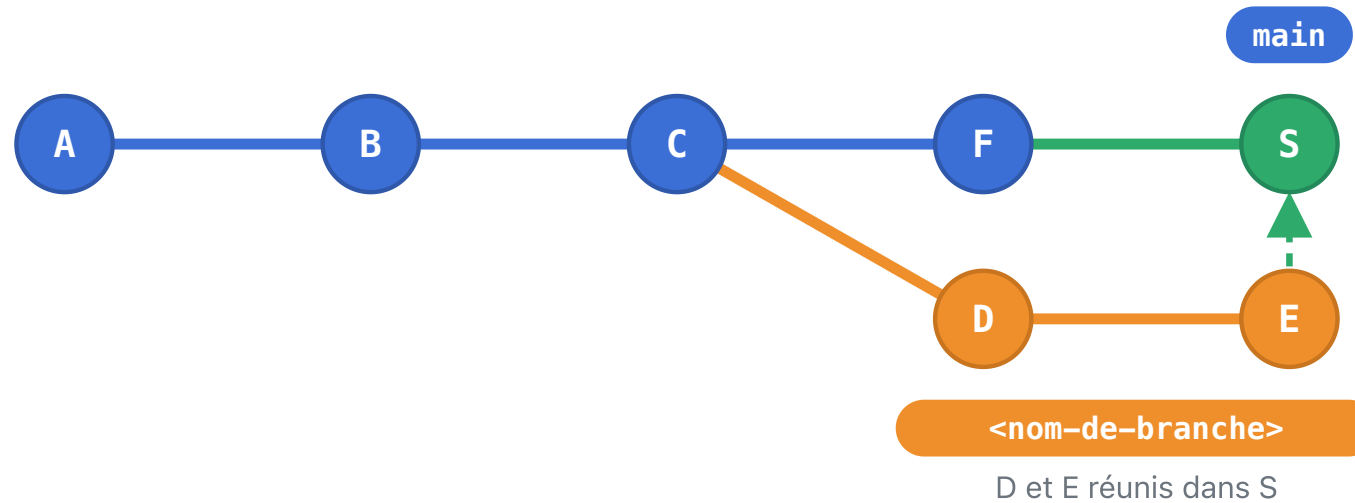
- Le binôme **relit** avant que le code n'entre dans `main`
- `main` garde **un commit par fonctionnalité** : historique lisible

Toujours dans cet ordre :

1. `git checkout -b presentation-ana`
2. modifier, `git add`, `git commit`
3. `git push -u origin presentation-ana`
4. GitHub : **Compare & pull request**, description en 3 lignes, *reviewer* = binôme
5. Le binôme relit : **Files changed**, commente, **Approve**
6. **Squash and merge**, puis **Delete branch**
7. `git checkout main` et `git pull`

Squash and merge

Squash and merge : un seul commit S sur main. La branche peut être supprimée.



- Les trois commits de la branche deviennent **un seul** commit **S** dans **main**
- La branche est supprimée : elle a servi
- Le bouton vert propose aussi *Merge* et *Rebase* : dans ce cours, **Squash and merge**

Quand ça coince

Push refusé

! [rejected] main -> main (fetch first)

Le binôme a poussé avant vous. Rien n'est cassé.

```
git pull
git push
```

Une fois pour toutes :

```
git config --global pull.rebase true
```

Conflit

Vous avez modifié **les mêmes lignes**. Git vous laisse choisir :

```
<<<<<<< HEAD
leur ligne
=====
votre ligne
>>>>>>> presentation-ana
```

1. garder ce qui doit rester, enlever les marqueurs
2. `git add main.cpp`
3. `git commit` (ou `git rebase --continue`)

Dans CLion : fenêtre *Conflicts*, bouton **Merge**, trois colonnes.

Les règles du cours

**Commit ne suffit pas : un commit reste sur votre ordinateur.
Personne ne le voit, rien n'est rendu. Push avant de partir !**

- Un exercice terminé = un commit, avec un message qui dit ce qu'il apporte : `Ajouter la présentation d'Ana`
- Ce qui est sur `GitHub` à la date de Roster est ce qui est vérifié
- Jamais de dossier de build (`cmake-build-debug`) ni `.idea` dans le dépôt : le `.gitignore` s'en charge
- SSH pour parler à GitHub, pas de mot de passe
- `git pull` avant de commencer, `git push` avant d'arrêter

Démo

1. Roster : accepter l'*assignment*, le dépôt apparaît sur GitHub
2. CLion : cloner avec l'URL **SSH**, *Trust project*, **Run**
3. Branche, une ligne de plus, commit, push
4. Pull request : titre, trois lignes, *reviewer*
5. Relecture : un commentaire, **Approve**
6. **Squash and merge**, **Delete branch**, historique sur GitHub
7. `git checkout main`, `git pull`
8. Le push refusé, en direct

À vous

Partie 1 (aujourd'hui)	Budget
0. Liste de contrôle du guide d'installation	10 min
1. Roster, clone, Run	15 min
2. Premier commit, premier push (A), pull (B)	15 min
3. Branche, pull request, conflit (chacun)	30 min
4. Le push refusé	10 min
5. Vérification sur GitHub	5 min

- L'étape 3 est la plus importante : faites-la en classe
- Guides d'accompagnement : *Installation*, *Git en pratique*, sur le site du cours
- Partie 2 vendredi : exercices et premières assertions